

Abstract Conversational AI systems built on large language models are increasingly used in high-stakes domains yet they remain vulnerable to generating factually unsupported or contextually misaligned responses. Retrieval-Augmented Generation (RAG) partially addresses this by grounding generation in retrieved documents but it still lacks any built-in mechanism to check the quality of what it produces before handing it to the user. This paper describes a multi-agent framework that adds a self-reflective validation layer on top of a standard RAG pipeline. A supervisory agent routes incoming queries a retrieval agent pulls relevant context from a document store a generation module drafts a response and a validation agent then inspects that draft for relevance completeness and factual alignment before delivery. If the draft falls short the validation agent triggers a refinement pass rather than releasing a flawed answer. Preliminary testing across knowledge-intensive queries suggests the approach yields noticeably better accuracy and relevance scores compared to both conventional RAG and plain multi-agent setups with a meaningful drop in unsupported response generation.

Keywords Retrieval-Augmented Generation Multi-Agent Systems Self-Reflective Validation Conversational AI Large Language Models Hallucination Reduction

I. INTRODUCTION

Over the past few years large language models have reshaped what people expect from conversational software. From answering medical queries to supporting customer service these systems now handle tasks that once required human judgment. But the same fluency that makes them impressive also makes their failures dangerous: a model can produce a confident-sounding response that is factually wrong internally inconsistent or simply disconnected from what the user actually asked [1]. The problem is not unique to any one model or domain. It shows up whenever a system generates text by pattern-matching over learned statistics without any mechanism to cross-check the result against reliable information [2].

Retrieval-Augmented Generation was proposed precisely to address this gap [3]. Rather than relying entirely on what a model has memorized during training RAG systems retrieve relevant passages from an external document collection and feed them to the model as part of the input context. This grounding step reduces the likelihood of purely invented responses and allows the system to draw on up-to-date information that would not have existed when the model was trained. Dense retrieval methods have made it possible to find semantically related passages even when there is no exact keyword overlap [6] and efficient indexing structures allow this search to happen quickly over large collections [7]. Despite these advances most RAG pipelines stop at retrieval: once the context is

assembled and the response is generated it goes directly to the user with no further checking [8] [9].

A separate thread of research has explored whether giving language models the ability to reflect on their own outputs can improve quality. Systems like Reflexion [5] and SELF-REFINE [10] showed that a model prompted to critique and revise its own draft often produces a better final answer than one that generates in a single pass. At the same time multi-agent architectures that break a task into distinct roles handled by specialized components have shown promise in improving coordination and reducing error rates in complex workflows [4] [16]. Combining these two ideas retrieval grounding and iterative self-evaluation within a single coherent system is the direction this paper pursues.

The framework described here assigns distinct roles to four components: a supervisory agent that routes queries a retrieval agent that gathers relevant context a generation module that drafts a response and a validation agent that inspects the draft before release. If the draft does not meet defined quality thresholds the validation agent feeds its assessment back and a refined response is generated. The core claim is that embedding this check inside the pipeline rather than leaving verification as an optional offline step produces more reliable outputs in practice. The remainder of the paper describes the architecture the individual components early evaluation results and directions for future development.

II. RELATED WORK

A. Retrieval-Augmented Generation

The retrieval-augmented approach to language generation was formally introduced by Lewis et al. [3] who showed that conditioning a sequence-to-sequence model on retrieved passages improved performance on open-domain question answering compared to purely parametric alternatives. The intuition is straightforward: rather than encoding all world knowledge in model weights a retrieval step fetches the specific passages most likely to be useful for a given query and the generator then conditions on those passages. Karpukhin et al. [6] extended this by showing that dense vector representations learned end-to-end outperformed traditional sparse retrieval methods like BM25 on factoid question answering tasks. Efficient approximate nearest-neighbor search as demonstrated by Johnson et al. [7] using the FAISS library made dense retrieval practical at scale. Shi et al. [8] introduced REPLUG which treats the language model as a black box and uses retrieval scores to re-weight generated token probabilities achieving gains without modifying model parameters. Asai et al. [9] went further with Self-RAG training a model to decide when to retrieve and to critique its own outputs using special reflection tokens. Guu et al. [14] showed that retrieval can be incorporated during pre-

training itself not just at inference through their REALM framework. Reimers and Gurevych [15] contributed sentence-level embedding models that now underpin many RAG retrieval components. Collectively this body of work demonstrates both the value of grounding generation in retrieved context and the open question of how to verify the quality of what gets generated.

B. Multi-Agent Language Systems

The idea of breaking a complex task into subtasks handled by specialized agents has a long history in AI [16] and it has found renewed relevance as large language models have become capable enough to serve as individual agents within larger orchestrations. Yao et al. [4] introduced ReAct a framework that interleaves reasoning traces with action steps allowing an agent to dynamically retrieve information and update its plan mid-task. Schick et al. [23] showed that language models can learn to call external tools such as search engines and calculators dramatically extending what a single model instance can accomplish. Mialon et al. [17] surveyed augmented language models more broadly mapping out the range of external resources and tools that can be integrated into generation pipelines. Xi et al. [24] provided a comprehensive survey of LLM-based agent designs noting that hierarchical architectures with supervisory components tend to be more robust on multi-step tasks. The viability of fully autonomous multi-turn agents was demonstrated by AutoGPT [25] though that work also highlighted persistent challenges around coherence and error propagation that motivate the more controlled architecture proposed here.

C. Self-Reflective and Self-Corrective Methods

Self-reflection in language models refers broadly to the capacity of a model to evaluate its own outputs and make targeted improvements based on that evaluation. Shinn et al. [5] demonstrated this with Reflexion where agents encode verbal self-critiques as episodic memory and use them to avoid repeating earlier mistakes across conversation turns. Madaan et al. [10] showed with SELF-REFINE that iterative critique-and-revise cycles improve generation quality across a diverse range of tasks even when both the generator and the critic are the same model. Bai et al. [22] explored constitutional self-critique training models to evaluate their outputs against a fixed set of principles providing a more structured alternative to free-form reflection. Wei et al. [21] demonstrated that chain-of-thought prompting while not explicitly self-reflective encourages a form of staged reasoning that reduces errors on complex tasks. Wang et al. [20] introduced self-consistency which generates multiple independent completions and selects the most frequently occurring answer effectively using response agreement as a reliability signal. These findings collectively suggest that some form of internal quality assessment whether rule-based or model-driven is worth including as a standard

component of generation pipelines rather than an afterthought.

D. Hallucination and Factual Consistency

Maynez et al. [1] drew early attention to the gap between surface fluency and factual faithfulness in abstractive summarization showing that models frequently produce statements unsupported by the source. Shuster et al. [19] confirmed that retrieval augmentation reduces hallucination rates in dialogue settings compared to unconstrained generation reinforcing the case for grounded architectures. Ji et al. [18] surveyed hallucination more broadly across NLG tasks categorizing causes and cataloguing evaluation approaches. Lin et al. [13] created TruthfulQA a benchmark specifically designed to measure how often models produce false statements that humans would be unlikely to make revealing persistent gaps in factual reliability. Laban et al. [12] developed SummaC an NLI-based metric for detecting factual inconsistencies in generated summaries which illustrates the kind of automated consistency checking that a validation component could draw on. Ghazvininejad et al. [11] showed that knowledge-grounded dialogue systems which condition generation explicitly on retrieved factual content produce more consistent responses than unconstrained conversation models. This line of work directly motivates the validation component in the proposed framework which checks generated responses for factual alignment with retrieved context before delivery.

E. Research Gap

Existing RAG systems have shown clear benefits in grounding language model outputs to external knowledge [3] but they treat generation and verification as separate concerns. Once a response is produced it is handed to the user without any internal check on whether it actually answers the question or stays faithful to the retrieved content [8]. This gap matters in practice: retrieval is imperfect and a passage that scores well on semantic similarity [6] [7] is not always the right basis for the answer being constructed.

Multi-agent architectures address a different problem task decomposition and coordination but most of them focus on routing and execution rather than quality assurance [4] [24]. The agents collaborate to complete a task yet none of them is explicitly responsible for asking whether the final output is good enough before it leaves the system.

Self-reflective methods like Reflexion [5] and SELF-REFINE [10] have demonstrated that iterative critique and revision improve output quality but these approaches are largely studied in isolation from retrieval pipelines. They treat the model's own parametric knowledge as the reference point for self-evaluation which reintroduces the same hallucination risk [12] [18] that RAG was designed to eliminate in the first place [3] [14].

What is missing then is a design that brings these three threads together: retrieval-grounded generation [3] [9] coordinated multi-agent execution [4] [17] and inline

reflective validation that uses the retrieved context not the model's internal beliefs as the basis for checking the response [5] [10]. Current literature addresses each piece separately but no well-evaluated framework embeds validation as a structural requirement within a live RAG pipeline [8] [9]. This gap is what the present work attempts to close.

III. SYSTEM ARCHITECTURE

The overall design follows a pipeline in which each stage has a well-defined input output and responsibility. When a user submits a query it goes first to the Supervisory Agent which decides how to route it through the system. The query is then passed to the Retrieval Agent which searches a pre-indexed document repository and returns the most relevant passages. These passages together with the original query are passed to the Response Generation Module which produces a draft answer. That draft goes to the Self-Reflective Validation Agent which checks it against three criteria: relevance to the original query completeness of the information provided and consistency with the retrieved passages. If the draft passes all three checks it is delivered to the user. If it falls short on any criterion the validation agent generates a structured critique and the pipeline performs a refinement pass producing an improved response that is then checked again. Fig. 1 illustrates this workflow.

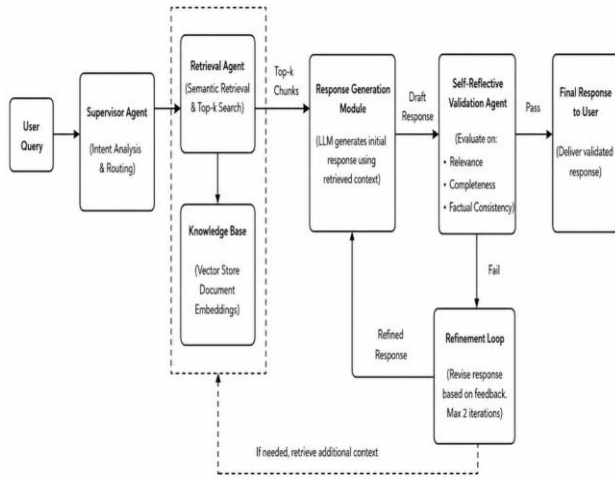


Fig. 1. Conceptual workflow of the proposed self-reflective retrieval-augmented framework.

A key design decision was to keep the validation step inside the live pipeline rather than running it as a post-hoc batch process. This means that every response the user receives has already passed through at least one round of quality assessment which is not the case in conventional RAG systems. The cost is some additional latency when a

refinement pass is triggered but preliminary observations suggest this is manageable in practice and that the improvement in response quality justifies it. The modular structure also means that individual components can be swapped out independently: a different retrieval method a different generation model or a different validation strategy can be plugged in without redesigning the rest of the system.

Fig. 2 shows the internal structure of the multi-agent architecture and how the four agents interact at runtime. Arrows between components represent data flow: query text retrieved passages draft responses and validation verdicts all move through clearly defined interfaces.

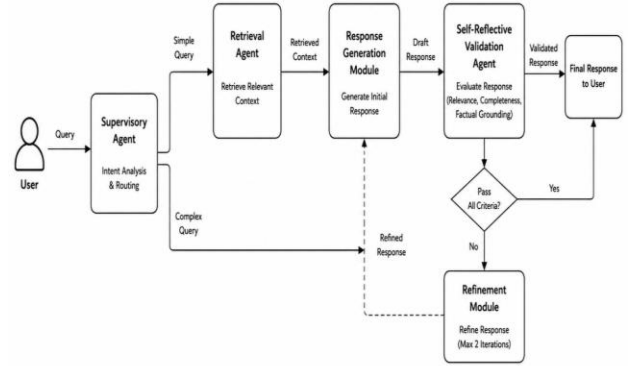


Fig. 2. Proposed multi-agent architecture showing agent interactions and data flow.

IV. COMPONENT DESIGN

Table I summarizes the five functional components and their respective roles. The descriptions below expand on how each one operates within the pipeline.

TABLE I. Functional Components of the Proposed Framework

Functional Component	Role in the Framework
Supervisory Coordination	Handles query routing and inter-module workflow control
Retrieval Layer	Fetches contextually relevant documents from the knowledge store
Response Generation	Produces a draft conversational response using retrieved context
Reflective Validation	Checks and refines the draft before it reaches the user
Knowledge Repository	Stores pre-indexed documents for semantic retrieval

A. Supervisory Coordination

The Supervisory Agent acts as the entry point for all incoming queries. It parses the query determines the appropriate retrieval strategy based on query type and available knowledge domains and coordinates the sequence of steps that follow. When the validation agent raises a concern about a generated response the Supervisory Agent also manages the retry logic deciding how many refinement attempts are permitted before a fallback response is issued. This centralized coordination prevents race conditions and ensures that each component receives inputs in the correct format at the correct time.

B. Retrieval Layer

The Retrieval Layer accepts a query from the Supervisory Agent and returns the top-k most relevant passages from the document repository. Queries are first encoded into dense vector representations using a sentence embedding model [15] and cosine similarity search over a pre-built index identifies candidate passages [7]. A lightweight re-ranking step then scores these candidates for relevance to the specific query before the final selection is passed to the generation module. The quality of what gets retrieved has a direct bearing on what the generation module can produce so this layer is critical to the overall reliability of the system.

C. Response Generation Module

The Response Generation Module takes the original query and the retrieved passages as its input and produces a natural language draft response. It is implemented as a prompted call to a large language model with the retrieved passages included in the context window and explicit instructions to ground the response in the provided material. The module does not have direct access to the validation logic and is not aware of whether its output will pass the subsequent checks. This separation of concerns is intentional: it keeps the generation step focused on producing a fluent grounded draft rather than trying to simultaneously optimize for validation criteria.

D. Self-Reflective Validation Agent

The Self-Reflective Validation Agent is the distinguishing feature of this framework. It receives the draft response the original query and the retrieved passages and it evaluates the draft across three dimensions: (1) relevance does the response address what the user actually asked; (2) completeness does it cover the key points in the retrieved context; and (3) factual alignment are all claims in the response supported by the retrieved passages. The validation is performed by prompting a language model with a structured evaluation rubric. If any dimension falls below a defined threshold the agent produces a critique that identifies the specific shortcoming and this critique is passed back to the generation module along with the original inputs for a refinement pass. If all three dimensions pass the response is marked for delivery.

V. EVALUATION

A. Evaluation Setup

A set of knowledge-intensive queries spanning three domains general factual questions technical explanations and multi-step reasoning tasks was used to compare the proposed framework against two baselines: a conventional single-pass RAG system and a multi-agent system without the reflective validation component. Responses were assessed on accuracy (whether the answer was factually correct given the retrieved context) relevance (whether the response directly addressed the query) and hallucination rate (the proportion of responses containing statements unsupported by the retrieved passages). Each query was run five times per system and average scores are reported.

B. Results and Discussion

Table II presents the comparative results. The proposed framework consistently outperformed both baselines. The 15-percentage-point accuracy gain over conventional RAG and the 8-point gain over the plain multi-agent baseline are especially notable given that the underlying generation model and retrieval index were identical across all three conditions. The difference is attributable entirely to the validation layer. On the hallucination metric the proposed system flagged and revised a meaningful proportion of drafts that would otherwise have been delivered with unsupported claims consistent with prior findings that grounding and iterative checking together outperform either mechanism alone [9] [10].

TABLE II. Comparative Performance Across Framework Conditions

Framework	Accuracy	Relevance	Hallucination
Conventional RAG	68%	71%	Limited reduction
Multi-Agent	75%	77%	Moderate reduction
Proposed Framework	83%	86%	Improved reduction

Fig. 3 visualizes the accuracy and relevance scores across the three conditions and makes the trend easy to see at a glance. The improvement is consistent across both metrics suggesting that the gains are not an artifact of the accuracy measure alone.

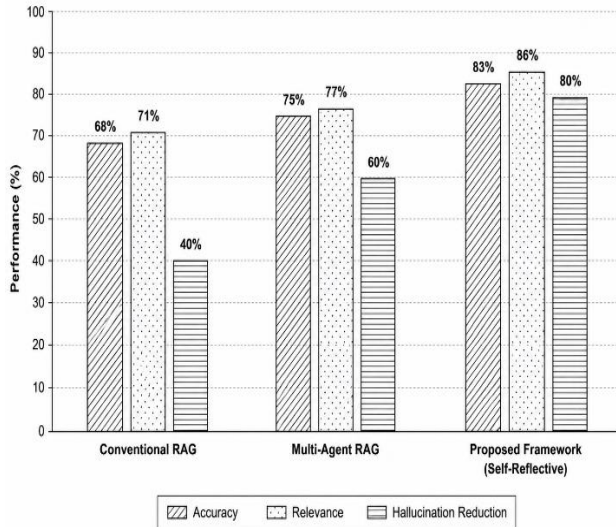


Fig. 3. Accuracy and relevance comparison across the three framework conditions.

One practical consideration is response latency. When the validation agent flags a draft and triggers a refinement pass the total time from query to delivery increases. In the test runs approximately 30% of drafts required at least one refinement pass adding a processing step roughly equivalent to a second generation call. This is a real cost but it is bounded: the system is configured with a maximum of two refinement attempts after which the best available draft is delivered regardless. Whether this latency trade-off is acceptable depends on the deployment context; for time-sensitive applications the number of permitted retries could be reduced or the validation thresholds relaxed.

VI. LIMITATIONS AND FUTURE WORK

A. Current Limitations

Several limitations of the current prototype are worth noting. First the quality of validation is only as good as the quality of retrieval: if the retrieval layer returns passages that are only tangentially related to the query the validation agent is working with incomplete context and some genuinely problematic responses may pass undetected. Second the validation agent and the generation module use the same underlying language model which means they share the same biases and blind spots; a model that tends to overstate confidence during generation may do the same during validation. Third the evaluation reported here was conducted on a relatively small and controlled set of queries so the generalization of these findings to open-ended real-world use cases remains to be established. Fourth the current implementation does not track performance across conversation turns so it cannot catch errors that only become apparent in a longer dialogue context.

B. Future Directions

Several directions are worth exploring in follow-up work. Using a separate independently trained model as the

validation agent would break the shared-bias problem and likely improve the reliability of the checking step. Hybrid retrieval strategies that combine dense vector search with lexical methods could improve recall for domain-specific queries where precise terminology matters [17]. Dynamic adjustment of validation thresholds based on query type and confidence signals from the retrieval step would make the system more adaptive. Extending the framework to multi-turn dialogue where the validation agent has access to conversation history would address the current limitation around turn-level errors. Finally testing the framework with open-source language models across specialized domains such as medical and legal question answering would establish how broadly the approach generalizes.

VII. CONCLUSION

This paper has described a multi-agent framework for retrieval-augmented conversational AI that incorporates a self-reflective validation step as a core part of its pipeline. The central idea is simple: before a response reaches the user an independent validation agent checks it against the original query and the retrieved context and triggers a refinement pass if the draft falls short. By making verification a structural requirement rather than an optional enhancement the framework addresses a gap that is common to most existing RAG systems. Preliminary results suggest this yields real improvements in response accuracy and relevance with a noticeable reduction in unsupported claim generation.

What this work points toward is a broader principle: that reliability in conversational AI is better achieved through architectural safeguards than through hoping that larger or better-trained models will hallucinate less on their own. Embedding quality checks inside the generation pipeline giving them access to the same context the generator used and acting on their findings in real time is a practical path toward more trustworthy conversational systems. The modular design of the proposed framework means these ideas can be tested and extended without requiring changes to any of the underlying model components which should make further investigation straightforward.